Software Development Lifecycle (SDLC)

CSYE 6225 - Boston

Spring 2026

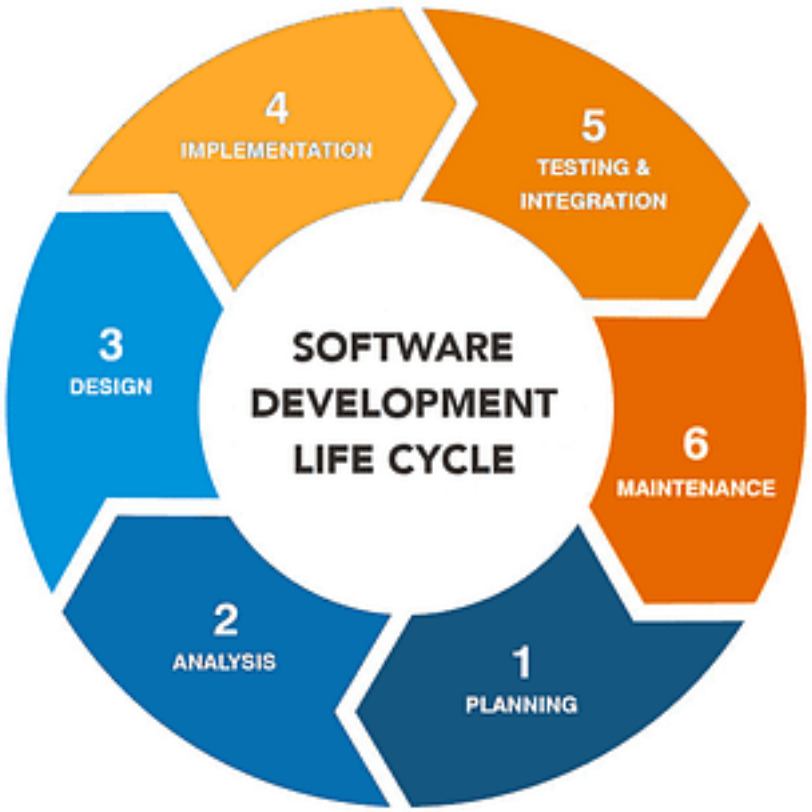
Tejas Parikh (t.parikh@northeastern.edu)

NORTHEASTERN.

Introduction

What is the SDLC?

- The Software Development Lifecycle is a structured process defining the stages involved in bringing software from conception to deployment and maintenance.
- It provides a systematic approach to building software that meets quality standards and user requirements.



Why It Matters for Cloud-Native

- In cloud-native environments, the SDLC takes on additional significance.
- Microservices, containerization, and continuous deployment pipelines mean each phase must account for scalability, resilience, and automation from the outset.

Core Phases of the SDLC

Phase 1: Planning and Requirements

- Establishes what the software should accomplish through stakeholder interviews, feasibility assessment, risk analysis, and resource planning.
- *Cloud-Native Consideration:* Planning must account for infrastructure costs, service boundaries, and deployment strategies early.

Phase 2: Design

- Translates requirements into architecture—deciding between monolithic vs. microservices, selecting databases, designing APIs, and establishing security patterns.
- *Cloud-Native Consideration:* Design decisions should embrace Twelve-Factor App principles, particularly statelessness and configuration management.

Phase 3: Implementation

- Developers write code according to design specifications using version control, code reviews, coding standards, and unit testing.
- *Cloud-Native Consideration:* Code should be container-ready with externalized configuration.

Phase 4: Testing

- Verifies software through unit testing (components), integration testing (interactions), system testing (end-to-end), and acceptance testing (user validation).
- *Cloud-Native Consideration:* Must include container orchestration scenarios and chaos engineering.

Phase 5: Deployment

- Moves software to production using strategies like blue-green deployments, canary releases, rolling updates, and feature flags.
- *Cloud-Native Consideration:* CI/CD pipelines and Infrastructure as Code are essential.

Phase 6: Maintenance and Operations

- Ongoing monitoring, incident response, performance tuning, and technical debt management.
- *Cloud-Native Consideration:* Observability through distributed tracing, centralized logging, and metrics becomes critical.

SDLC Models

Waterfall

- Sequential phases where each must complete before the next begins. Useful for stable, well-defined requirements but criticized for inflexibility.

Agile

- Emphasizes collaboration, adaptability, and frequent delivery. Responds to change, measures progress through working software, and maintains sustainable pace.

DevOps and Continuous Delivery

- Integrates development and operations through CI/CD, Infrastructure as Code, and Site Reliability Engineering practices.
- Extends Agile with automation and feedback loops.

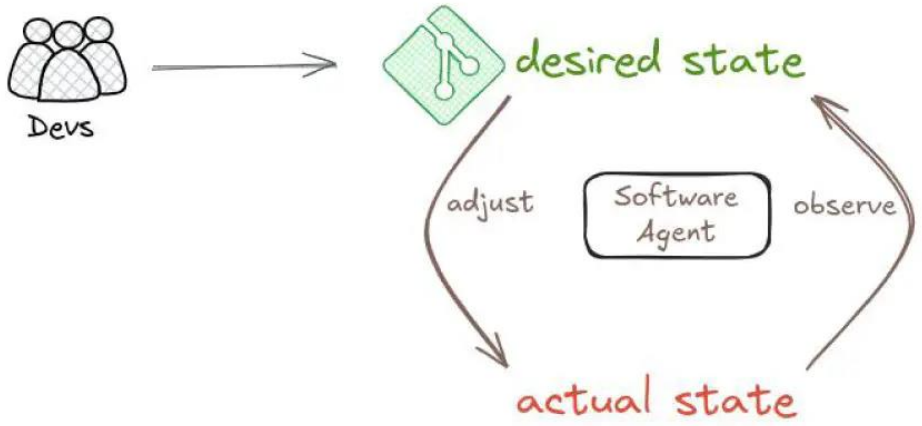
Cloud-Native SDLC Principles

Shift-Left

- Testing, security, and operational concerns move earlier in the lifecycle.
- Security scanning in CI pipelines, performance testing during development, and infrastructure validation before merge.

GitOps

- Git becomes the source of truth for both application code and infrastructure.
- Changes flow through pull requests with automated validation.



Independent Service Lifecycles

- Each microservice can have its own SDLC cadence.
- Teams own services end-to-end, from design through operations.

The Role of Observability

- Traditional monitoring evolves into observability: the ability to understand system behavior through external outputs.
- This supports faster debugging and proactive maintenance.

Key Takeaways

Choosing the Right Model

- Selection depends on project complexity, requirement stability, team experience, compliance needs, and organizational culture.

Common Pitfalls

- Teams often struggle with insufficient requirements gathering leading to scope creep, skipping design in favor of immediate coding, inadequate testing coverage, manual deployment processes prone to error, and neglecting operational concerns until production issues arise.

Resources

Additional resources can be found on the lecture page